

Scenario Based Questions

Question 1:

You are tasked with designing a program to simulate a library management system. In this system, the following requirements must be fulfilled:

1) Create a class Library with the following attributes:

- name (name of the library).
- books (a dictionary storing book titles as keys and their quantities as values).

2) Create a child class DigitalLibrary that inherits from the Library class:

- Includes a digital_books attribute (stores a list of digital book titles available for online access).

3) Implement the following methods:

- In the Library class:
- add_book(title, quantity): Adds a book and its quantity to the library.
- borrow_book(title): Allows a user to borrow a book, reducing its quantity.
- return_book(title): Allows a user to return a borrowed book, increasing its quantity.
- In the DigitalLibrary class:
- access_digital_book(title): Checks if the book is available digitally and allows access if available.

4) Demonstrate the following scenario:

- Add 5 copies of "Python Programming" and 3 copies of "Data Science 101" to the library.
- Borrow 2 copies of "Python Programming."
- Access the digital version of "Data Science 101."
- Return 1 copy of "Python Programming."
- Display the remaining physical and digital books in the library.

Requirements:

- Demonstrate the concepts of inheritance, encapsulation, and object interactions.
- Handle edge cases (e.g., trying to borrow a book not available in sufficient quantity).
- Provide the full implementation in Python.

Question 2:

You are tasked with designing a program to simulate a hotel reservation system. In this system, the following requirements must be fulfilled:

1) Create a class `HotelRoom` with the following attributes:

- `room_type` (e.g., Single, Double).
- `price_per_night`.
- `is_available` (boolean indicating if the room is available).

2) Create child classes `DeluxeRoom` and `Suite` with:

- Additional attributes (e.g., complimentary services for each room type).

3) Implement the following methods:

- In the `HotelRoom` class:
 - `book_room(nights)`: Books the room for a given number of nights if it's available.
 - `cancel_booking()`: Cancels the booking.
 - `display_details()`: Displays room details.
- In the child classes:
 - Override `display_details()` to include additional features.

4) Demonstrate the following scenario:

- Book a "DeluxeRoom" for 3 nights.
- Display the details of the "Suite."
- Cancel the booking for the "DeluxeRoom."
- Handle edge cases like trying to book an already-booked room.

Requirements:

- Demonstrate inheritance, polymorphism, and encapsulation.
- Ensure methods handle invalid input (e.g., negative number of nights).
- Provide the full implementation in Python.

Question 3:

You are tasked with designing a program to simulate an e-commerce platform. In this system, the following requirements must be fulfilled:

1) Create a class `ECommerce` with the following attributes:

- products (a dictionary of product names and their prices).
- cart (a dictionary of items added to the cart).

2) Implement the following methods:

- add_to_cart(product, quantity): Adds a product to the cart.
- remove_from_cart(product): Removes a product from the cart.
- checkout(): Calculates the total cost and clears the cart.

3) Demonstrate the following scenario:

- Add “Laptop” (price: \$1000) and “Headphones” (price: \$200) to the products list.
- Add 1 Laptop and 2 Headphones to the cart.
- Remove 1 Headphone from the cart.
- Checkout and display the total amount.

Requirements:

- Demonstrate the concepts of classes, methods, and object interactions.
- Ensure methods handle edge cases (e.g., removing an item not in the cart).
- Provide the full implementation in Python.

Question 4:

You are tasked with designing a program to simulate a banking system. In this system, the following requirements must be fulfilled:

1) Create a class BankAccount with the following attributes:

- account_holder (name of the account holder).
- balance (account balance).

2) Create child classes SavingsAccount and CurrentAccount:

- SavingsAccount: Adds an interest_rate attribute.
- CurrentAccount: Adds an overdraft_limit attribute.

3) Implement the following methods:

- deposit(amount): Adds funds to the account.
- withdraw(amount): Deducts funds, considering overdraft or interest.
- display_balance(): Displays the balance.

4) Demonstrate the following scenario:

- Create a SavingsAccount for Alice with an interest rate of 5%.
- Deposit \$1000 into Alice's account.
- Withdraw \$200 from Alice's account.
- Display Alice's balance.
- Create a CurrentAccount for Bob with an overdraft limit of \$500.
- Withdraw \$1200 from Bob's account.
- Display Bob's balance.

Requirements:

- Demonstrate inheritance, encapsulation, and method overriding.
- Ensure methods handle edge cases (e.g., withdrawing more than the overdraft limit).
- Provide the full implementation in Python.

Question 5:

You are tasked with designing a program to simulate an event ticket booking system. In this system, the following requirements must be fulfilled:

1) Create a class Event with the following attributes:

- event_name (name of the event).
- ticket_price (price of a single ticket).
- tickets_available (number of tickets available for the event).

2) Implement the following methods:

- In the Event class:
- book_ticket(quantity): Allows a user to book a specific number of tickets for the event. Deduct the booked tickets from the total available tickets and display the total cost of the booking.
- cancel_ticket(quantity): Allows a user to cancel a specific number of tickets they have previously booked. Add the canceled tickets back to the total available tickets and refund the cost of the canceled tickets.
- display_event(): Displays the event name, ticket price, and the number of tickets available.

3) Demonstrate the following scenario:

- Create an event called “Music Concert” with a ticket price of \$50 and 100 tickets available.
- A user books 10 tickets for the “Music Concert.”
- The same user cancels 3 tickets.
- Display the details of the “Music Concert” after these operations.

Requirements:

- Demonstrate the concepts of classes, attributes, and methods.
- Ensure that all operations handle edge cases (e.g., attempting to book more tickets than available or canceling more tickets than booked).
- Write the full implementation in Python.

Question 6:

You are tasked with designing a program to simulate a FinTech system in Pakistan, focusing on digital wallets and banking solutions. The system works as follows:

1) Create a base class DigitalWallet with the following attributes:

- wallet_name (name of the digital wallet, e.g., “EasyPaisa”, “JazzCash”).
- balance (the current balance in the wallet).
- transaction_limit (the maximum amount that can be transferred in a single transaction).

2) Create two child classes BankWallet and MobileWallet that inherit from the DigitalWallet class:

- BankWallet: Represents a wallet linked to a bank account. Adds an attribute linked_bank (name of the linked bank) and a method for bank-to-wallet transfers.
- MobileWallet: Represents a wallet based on mobile numbers. Adds an attribute mobile_number (user’s mobile number) and a method for mobile wallet-to-wallet transfers.

3) Implement the following methods:

- In the DigitalWallet class:
 - deposit(amount): Adds funds to the wallet.
 - withdraw(amount): Deducts funds from the wallet if sufficient balance is available.
 - display_balance(): Displays the wallet’s balance.
- In the BankWallet class:
 - bank_transfer(amount): Transfers an amount from the linked bank to the wallet, ensuring it does not exceed the transaction limit.

- In the MobileWallet class:
- `mobile_transfer(receiver_wallet, amount)`: Transfers an amount to another mobile wallet, deducting it from the sender's wallet.

4) Demonstrate the following scenario:

- Create a BankWallet instance for "EasyPaisha" linked to "HBL" with a balance of PKR 10,000 and a transaction limit of PKR 25,000.
- Deposit PKR 5,000 into the EasyPaisha wallet and display the balance.
- Perform a bank-to-wallet transfer of PKR 15,000.
- Create a MobileWallet instance for "JazzCash" with a mobile number "+923001234567" and a balance of PKR 7,000.
- Transfer PKR 3,000 from the JazzCash wallet to the EasyPaisha wallet.
- Display the updated balances for both wallets.

Requirements:

- Demonstrate the concepts of inheritance, method overriding, encapsulation, and object interactions.
- Ensure all methods handle edge cases (e.g., insufficient balance, exceeding transaction limits, or invalid inputs).
- Write the full implementation in Python.

Question 7:

You are tasked with designing a program to simulate an agriculture supply chain system in Pakistan. The system works as follows:

1) Create a base class SupplyChain with the following attributes:

- `product_name` (name of the agricultural product, e.g., "Wheat", "Rice").
- `price_per_kg` (price of the product per kilogram).
- `quantity_available` (total quantity of the product available in kilograms).

2) Create two child classes Farmer and Vendor that inherit from the SupplyChain class:

- Farmer: Represents the farmer supplying the product. Adds an attribute `location` (geographic location of the farmer).
- Vendor: Represents the vendor selling the product. Adds an attribute `shop_name` (name of the vendor's shop).

3) Implement the following methods:

- In the SupplyChain class:
- supply_product(quantity): Reduces the available quantity when the farmer supplies a certain amount to the vendor.
- purchase_product(quantity): Deducts the purchased quantity from the available stock and calculates the total cost.
- display_details(): Displays the product's name, price per kilogram, and available quantity.
- In the Farmer class:
- add_stock(quantity): Adds more stock to the farmer's supply and updates the total quantity available.
- In the Vendor class:
- sell_product(quantity): Allows the vendor to sell the product to customers, reducing the stock.

4) Demonstrate the following scenario:

- Create a Farmer instance for a farmer located in "Multan" supplying "Wheat" at PKR 50 per kg with an initial stock of 10,000 kg.
- The farmer supplies 2,000 kg of wheat to a vendor.
- Create a Vendor instance for a vendor running a shop named "Karachi Grain Market" with an initial stock of 0 kg.
- The vendor sells 1,000 kg of wheat to a customer.
- Display the updated stock for both the farmer and the vendor after these operations.

Requirements:

- Demonstrate the concepts of inheritance, method overriding, encapsulation, and object interactions.
- Ensure all methods handle edge cases (e.g., attempting to supply or purchase more than available stock).
- Write the full implementation in Python.

Question 8:

You are tasked with designing a program to simulate a housing society management system in Pakistan, focusing on property ownership and maintenance. The system works as follows:

1) Create a base class Property with the following attributes:

- property_id (unique identifier for the property).
- owner_name (name of the property owner).
- size (size of the property in square yards).
- monthly_maintenance_fee (maintenance fee charged for the property).

2) Create two child classes ResidentialProperty and CommercialProperty that inherit from the Property class:

- ResidentialProperty: Adds an attribute number_of_rooms (number of rooms in the property).
- CommercialProperty: Adds an attribute business_type (type of business operating on the property).

3) Implement the following methods:

- In the Property class:
 - update_owner(new_owner): Updates the owner's name for the property.
 - get_owner(): Returns the current owner's name.
 - calculate_annual_fee(): Calculates and returns the total annual maintenance fee.
- In the ResidentialProperty class:
 - get_property_info(): Displays the property's details, including the number of rooms.
- In the CommercialProperty class:
 - get_property_info(): Displays the property's details, including the business type.

4) Demonstrate the following scenario:

- Create a ResidentialProperty instance with property_id "R123", owned by "Ali Khan", of size 500 square yards, with 3 rooms and a monthly maintenance fee of PKR 2,000.
- Create a CommercialProperty instance with property_id "C456", owned by "Sara Malik", of size 1000 square yards, operating as a "Retail Store", and a monthly maintenance fee of PKR 5,000.
- Change the owner of the residential property to "Zara Ahmed".
- Display the current owner of both properties using the get_owner() method.
- Calculate and display the total annual maintenance fee for both properties.
- Display all details of the properties using the get_property_info() method.

Requirements:

- Demonstrate the concepts of inheritance, encapsulation, method overriding, and the use of getter methods.
- Ensure all methods handle edge cases (e.g., invalid property IDs or fees).
- Write the full implementation in Python.

Question 9:

You are tasked with designing a program to simulate a hospital management system in Pakistan. The system works as follows:

1) Create a base class Hospital with the following attributes:

- hospital_name (name of the hospital, e.g., "Agha Khan").
- location (city or area of the hospital).
- total_beds (total number of beds available in the hospital).
- occupied_beds (number of beds currently occupied).

2) Create two child classes Doctor and Patient that inherit from the Hospital class:

- Doctor: Adds attributes doctor_id (unique ID for the doctor) and specialization (field of medical expertise, e.g., "Cardiology").
- Patient: Adds attributes patient_id (unique ID for the patient) and illness (description of the illness).

3) Implement the following methods:

- In the Hospital class:
 - admit_patient(): Admits a patient by increasing the count of occupied_beds and checking if beds are available.
 - discharge_patient(): Discharges a patient by decreasing the count of occupied_beds.
 - get_bed_availability(): Returns the number of beds available in the hospital.
- In the Doctor class:
 - get_doctor_info(): Returns the doctor's ID and specialization.
- In the Patient class:
 - get_patient_info(): Returns the patient's ID and illness.

4) Demonstrate the following scenario:

- Create a Hospital instance for "Shifa International" in Islamabad with 200 total beds and 150 currently occupied.
- Admit 3 new patients into the hospital.
- Create a Doctor instance for Dr. "Ayesha Khan" with ID "D101" specializing in "Neurology."
- Create a Patient instance for a patient named "Ali Raza" with ID "P501" suffering from "Migraine."
- Display the available beds in the hospital after admitting the patients.
- Display the details of the doctor using get_doctor_info() and the patient using get_patient_info().

Requirements:

- Demonstrate the concepts of inheritance, encapsulation, method overriding, and real-life object interaction.
- Ensure all methods handle edge cases (e.g., attempting to admit a patient when no beds are available).
- Write the full implementation in Python.

Question 10:

You are tasked with designing a program to simulate an online grocery delivery system in Pakistan. The system works as follows:

1) Create a base class GroceryStore with the following attributes:

- store_name (name of the grocery store, e.g., "Imtiaz Super Market", "Metro").
- available_products (a dictionary of products available in the store with the product name as key and price per unit as value).
- delivery_fee (the delivery fee for the order).

2) Create two child classes Customer and DeliveryPerson that inherit from the GroceryStore class:

- Customer: Adds attributes customer_id (unique ID for the customer) and order_list (a list of products ordered with quantities).
- DeliveryPerson: Adds attributes delivery_person_id (unique ID for the delivery person) and assigned_orders (list of orders assigned to the delivery person).

3) Implement the following methods:

- In the GroceryStore class:
 - add_product(product_name, price): Adds a new product to the store's available products.
 - remove_product(product_name): Removes a product from the store's inventory.
 - calculate_order_cost(order): Calculates the total cost of the order, including the delivery fee.
- In the Customer class:
 - place_order(order): Allows the customer to place an order by adding products to the order_list.
- In the DeliveryPerson class:
 - assign_order(order): Assigns a new order to the delivery person and updates their assigned orders.
 - deliver_order(order): Marks the order as delivered and removes it from the assigned orders.

4) Demonstrate the following scenario:

- Create a GroceryStore instance for a store named "Metro" with a product list including "Rice" (PKR 100), "Wheat Flour" (PKR 150), and "Sugar" (PKR 80). The delivery fee is PKR 50.

- Create a Customer instance for a customer named “Ali” with customer_id “C1001”. The customer places an order for 2 kg of Rice and 3 kg of Wheat Flour.
- Create a DeliveryPerson instance for a delivery person named “Ahmed” with delivery_person_id “D2001”. The delivery person is assigned the order placed by the customer.
- Display the order cost (including delivery fee) and the products in the order.
- The delivery person delivers the order. Display the order status and the delivery person’s assigned orders after delivery.

Requirements:

- Demonstrate the concepts of inheritance, encapsulation, method overriding, and object interactions.
- Ensure all methods handle edge cases (e.g., invalid product names, exceeding available stock, or delivery issues).
- Write the full implementation in Python.

Question 11:

You are tasked with designing a program to simulate the course registration and grading system for FAST University’s BS FinTech program. The system works as follows:

1) Create a base class Student with the following attributes:

- student_id (unique identifier for each student).
- name (name of the student).
- degree_program (the degree program the student is enrolled in, e.g., “BS FinTech”, “BS Computer Science”).
- total_credits (total number of credits enrolled in for the current semester).
- grades (a dictionary where the keys are course names, and values are the grades obtained).

2) Create two child classes UndergraduateStudent and PostgraduateStudent that inherit from the Student class:

- UndergraduateStudent: Adds an attribute sgpa (Semester Grade Point Average) which is calculated based on the grades obtained in the current semester.
- PostgraduateStudent: Adds an attribute cgpa (Cumulative Grade Point Average) which is calculated based on the grades obtained throughout the entire degree program.

3) Implement the following methods:

- In the Student class:
- `choose_degree(degree_program)`: Allows the student to select their degree program (e.g., "BS FinTech").
- `register_course(course_name, credits)`: Allows the student to register a course for the current semester, adding it to their grades dictionary with an initial grade of "Not Graded".
- `update_grade(course_name, grade)`: Updates the grade of a particular course in the grades dictionary.
- the UndergraduateStudent class:
- `calculate_sgpa()`: Calculates and returns the Semester Grade Point Average (SGPA) based on the courses registered and their respective grades.
- In the PostgraduateStudent class:
- `calculate_cgpa()`: Calculates and returns the Cumulative Grade Point Average (CGPA) based on the courses registered and their respective grades for the entire degree program.

4) Demonstrate the following scenario:

- Create an UndergraduateStudent instance for a student named "Ahmed Ali" with student_id "S101". The student chooses the degree program "BS FinTech". Then, the student registers for 5 courses with the following credits: "Math 101" (3 credits), "FinTech 201" (4 credits), "Economics 101" (3 credits), "Accounting 102" (2 credits), and "Business Law 103" (3 credits). Initially, all grades are "Not Graded".
- Create a PostgraduateStudent instance for a student named "Sara Khan" with student_id "S102". The student chooses the degree program "MS FinTech" and has completed 3 semesters with the following course records: "Advanced FinTech" (4 credits, grade "A"), "Blockchain Technologies" (3 credits, grade "B+"), and "Data Science" (4 credits, grade "A").
- Both students update their grades as they receive results.
- Calculate and display the SGPA for "Ahmed Ali" and the CGPA for "Sara Khan".
- Display the course list, grades, and total credits for each student.

Requirements:

- Demonstrate the concepts of inheritance, encapsulation, method overriding, and object interactions.
- Ensure all methods handle edge cases (e.g., invalid course names, ungraded courses, and invalid grade inputs).
- Write the full implementation in Python.